

# Sliding Window Median (Optimal Solution) — Dry Run Guide

Goal: Understand the two-heaps + lazy deletion approach with a step-by-step visual walkthrough.

## Core Idea

We maintain two heaps:

- **small** (max heap): smaller half of numbers
- **large** (min heap): larger half of numbers

For odd k, median is top of **small**.

For even k, median is average of tops of both heaps.

Example:

Window = [1, 3, -1]

Sorted = [-1, 1, 3]

small (left half) = [-1, 1]

large (right half) = [3]

Median = 1

## Input Example

nums = [1, 3, -1, -3, 5, 3, 6, 7]

k = 3

### i=0, add 1

Window not formed yet

small = [1]

large = []

Median not computed (need 3 elements).

### i=1, add 3

small = [1]

large = [3]

Balanced:

left half = [1]

right half = [3]

Median not computed yet.

### i=2, add -1

Current window: [1, 3, -1]

Sorted: [-1, 1, 3]

small = [1, -1]

large = [3]

Diagram:

[-1, 1] | [3]  
small large

Median = top(small) = 1

### **i=3, add -3**

Current window should become [3, -1, -3]

Before removing expired:  
old outgoing = 1

Lazy deletion:  
mark 1 as expired.  
Remove later when it reaches heap top.

After balancing:  
Sorted window: [-3, -1, 3]

small = [-1, -3]  
large = [3]

Diagram:  
[-3, -1] | [3]

Median = -1

### **i=4, add 5**

Window: [-1, -3, 5]  
Sorted: [-3, -1, 5]

small = [-1, -3]  
large = [5]

Diagram:  
[-3, -1] | [5]

Median = -1

### **i=5, add 3**

Window: [-3, 5, 3]  
Sorted: [-3, 3, 5]

small = [3, -3]  
large = [5]

Diagram:  
[-3, 3] | [5]

Median = 3

### **i=6, add 6**

Window: [5, 3, 6]  
Sorted: [3, 5, 6]

small = [5, 3]  
large = [6]

Diagram:  
[3, 5] | [6]

Median = 5

### **i=7, add 7**

Window: [3, 6, 7]  
Sorted: [3, 6, 7]

small = [6, 3]

```
large = [7]
```

Diagram:

```
[3,6] | [7]
```

```
Median = 6
```

## Important Concepts

1. Why two heaps?

To access median quickly.

2. Why lazy deletion?

Removing random values from heap is slow.

Instead:

- mark expired values
- remove only when they reach heap top

3. Balance Rule

small can have:

- same size as large

OR

- exactly 1 extra element

4. Time Complexity

Each insert/remove =  $O(\log k)$

Total =  $O(n \log k)$